

UNITY DEVELOPER TECHNICAL TEST

2D Airplane Shooting Game

Engine	Position	Deadline	Contact
Unity 2022.3.62f2	Unity Developer	3 days from receipt	diendat@spiritbomb.co

1. Overview

Welcome to the **SpiritBomb** Unity Developer Technical Test. This challenge is designed to evaluate your ability to build a complete, polished mini-game from scratch — one that looks good, sounds great, and above all feels great to play.

We care deeply about game feel. A technically correct game that is lifeless will score lower than a slightly rougher game that is genuinely fun and engaging. We want to see that you understand what makes players smile: satisfying explosions, punchy sound effects, screen shake, particle bursts, and all the little details that separate a prototype from an experience.

AI tools are not just permitted — they are encouraged. We are a studio that embraces modern development practices, and we value candidates who know how to leverage AI intelligently and responsibly. However, we also value honesty: you must declare exactly where and how AI was used.

2. The Assignment

Build a classic 2D vertical-scrolling (or side-scrolling) airplane shooting game using Unity 2022.3.62f2. The game should be self-contained, buildable, and playable in a single sitting. Quality of experience is valued over quantity of features.

2.1 Core Gameplay Requirements

The following mechanics must be implemented and functional:

- Player-controlled airplane that moves smoothly in response to keyboard or mouse input
- Player shooting mechanic — bullets, lasers, or any projectile the player can fire
- Enemy spawning system with at least two distinct enemy types or behaviors
- Collision detection between projectiles, enemies, and the player

- Health / lives system for the player
- Score system that tracks and displays the player's progress
- A clear win condition and a game over state
- A main menu or start screen and an end-of-game screen (game over / victory)
- Basic HUD: score, health or lives remaining, and any power-up status

2.2 Optional — Bonus Features

These are not required, but any of the following will be viewed positively:

- Boss encounter with multiple phases
- Power-up or upgrade system
- Procedurally generated or wave-based enemy spawning
- Multiple levels or a progression system
- Parallax scrolling background
- High-score persistence across sessions
- Any other creative mechanic that improves the experience

3. Game Feel & Polish — Our Primary Focus

 **This section carries the highest evaluation weight (20%). A game that feels amazing will stand out far more than a game with more features that feels flat.**

Think of game feel as the emotional response your game produces in the player. Every hit, every explosion, every death should feel like it has weight and consequence.

We want to see that you are passionate about the player experience — not just the code that runs underneath it.

4. AI Tools Policy

 **AI tools are fully permitted and encouraged.**

We are a studio that works with AI daily. We actively want to hire developers who are comfortable using AI tools to accelerate their workflow, explore ideas, and solve problems.

There is no penalty for using AI. There is, however, a requirement for transparency.

4.1 Permitted Uses

- Code generation or autocompletion (GitHub Copilot, ChatGPT, Claude, Cursor, etc.)
- Architecture suggestions and design pattern recommendations
- Debugging assistance and error explanations
- Generating placeholder art, textures, or sprite concepts
- Writing shader code, particle system configurations, or audio compression scripts

- Creating or sourcing sound effects and music (AI-generated audio is acceptable)
- Writing or improving documentation and comments

4.2 Mandatory Disclosure — AI Usage Declaration

Your submission must include an AI Usage Declaration, either in the README or as a separate CREDITS.md file. This declaration must clearly state:

- Which AI tools you used (e.g., GitHub Copilot, ChatGPT 4o, Claude Sonnet, Midjourney)
- Which specific parts of the project were generated or significantly assisted by AI — be precise (e.g., “The enemy spawning system in EnemyManager.cs was generated by ChatGPT”)
- Which parts were written entirely by you without AI assistance
- A brief reflection (3–5 sentences) on how AI helped or hindered your workflow during this project

 **Failing to disclose AI usage, or misrepresenting the extent of AI involvement, will result in immediate disqualification.**

Honesty and self-awareness about your own workflow are qualities we value as much as technical skill.

5. Technical Requirements

- Engine: Unity 2022.3.62f2 (LTS) — this exact version is required. Do not use a newer or older release.
- Render Pipeline: Any (Built-in, URP, or HDRP) — URP is recommended for 2D projects.
- Platform Target: **Android** as the primary build target.
- Avoid third-party asset store packages that do the bulk of the gameplay work for you. Utility libraries (DOTween, TextMeshPro, etc.) are fine.
- Code must be in C#. No visual scripting or Bolt nodes.
- The project must include a **playable build (.apk)** inside a /Build folder, or clear instructions for creating one.
- The project must include a **short video gameplay** to demonstrate gameplay.

6. Evaluation Criteria

Your submission will be assessed across five dimensions. The table below shows how each area is weighted:

Category	Weight	What We Are Looking At
Core Gameplay	40%	Working mechanics, controls, enemy AI, win/lose states, fun factor
Game Feel & Polish	20%	Sound design, particle system, visual effects, juice feeling
Code Quality	20%	Readability, architecture, naming conventions, no unused code
Creativity & Bonus	10%	Vibrant and aesthetically pleasing UI/HUD displays. Unique mechanics, or a surprise that delights the player
AI Usage & Transparency	10%	Thoughtful, purposeful AI use with honest and detailed disclosure

Evaluators will play the game for a minimum of five minutes before reviewing the code. First impressions matter.

7. Code Quality Expectations

While game feel is our primary focus, code quality is still evaluated. We look for:

- Clear, self-documenting naming for classes, methods, and variables
- Sensible separation of concerns — avoid stuffing all logic into a single MonoBehaviour
- No commented-out dead code left in the project
- Prefab-driven design — enemies, projectiles, and effects should be prefabs, not hardcoded scene objects
- Solid understanding of systems thinking, design patterns, OOP, and SOLID principles, with demonstrated ability to apply them in a real-time game environment.

8. Submission Instructions

8.1 Deadline

You have 3 calendar days from the date you receive this document to submit your work. Submissions received after the deadline will not be considered unless an extension has been explicitly agreed in advance.

8.2 Submission Methods

Choose one of the following two methods:

Option A — GitHub Repository	Option B — Email ZIP
1. Create a public GitHub repository. 2. Push your complete Unity project (excluding the Library/ and Temp/ folders). 3. Include your Build/ folder, Video gameplay and README. 4. Invite diendat@spiritbomb.co as a collaborator. 5. Email the repository link to diendat@spiritbomb.co with the subject line: [Unity Test] Your Name.	1. ZIP your complete Unity project (excluding Library/ and Temp/). 2. Include your Build/ folder, Video gameplay and README inside the ZIP. 3. Name the file: UnityTest_YourName.zip 4. Email it to diendat@spiritbomb.co with the subject line: [Unity Test] Your Name. 5. If the file is too large for email, use Google Drive, Dropbox, or WeTransfer and share the link.

8.3 Required README Contents

Your README (Markdown format) must include all of the following:

- Project name and a one-paragraph description of your game
- How to open and run the project in Unity 2022.3.62f2
- Controls reference (keyboard/mouse layout)
- Known issues or incomplete features, if any
- AI Usage Declaration (see Section 4.2)
- Credits for any third-party assets, sounds, or fonts used, with their licenses

9. What We Are Really Looking For

Beyond the requirements above, we want to see someone who genuinely cares about making games that players enjoy. We are not expecting a commercial-quality product. We are expecting to see evidence of craft, curiosity, and care. A small, polished gem will always beat a sprawling, unfinished project.

10. Questions & Contact

If you have any questions about the requirements, clarifications on scope, or need to request a deadline extension, please reach out before you begin work — not at the last moment.

Email: diendat@spiritbomb.co

Subject line format: [Unity Test] Your Name — Question

Good luck — and most importantly, have fun making it.

We are rooting for you.
